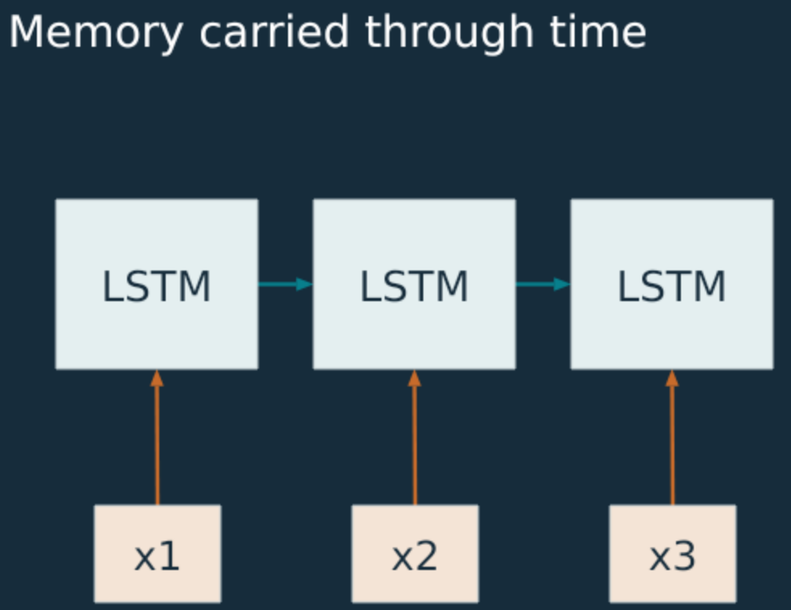
Long short-term memory networks

A tutorial in sequence order, gated memory, and working code

Sequence order and shared recurrent weights
Cell state, hidden state, and gates
Training, evaluation, and prediction



20 slides with detailed notes
Python + NumPy laboratory included

A sequence has a time axis

Reordering observations can change the correct prediction.

Value

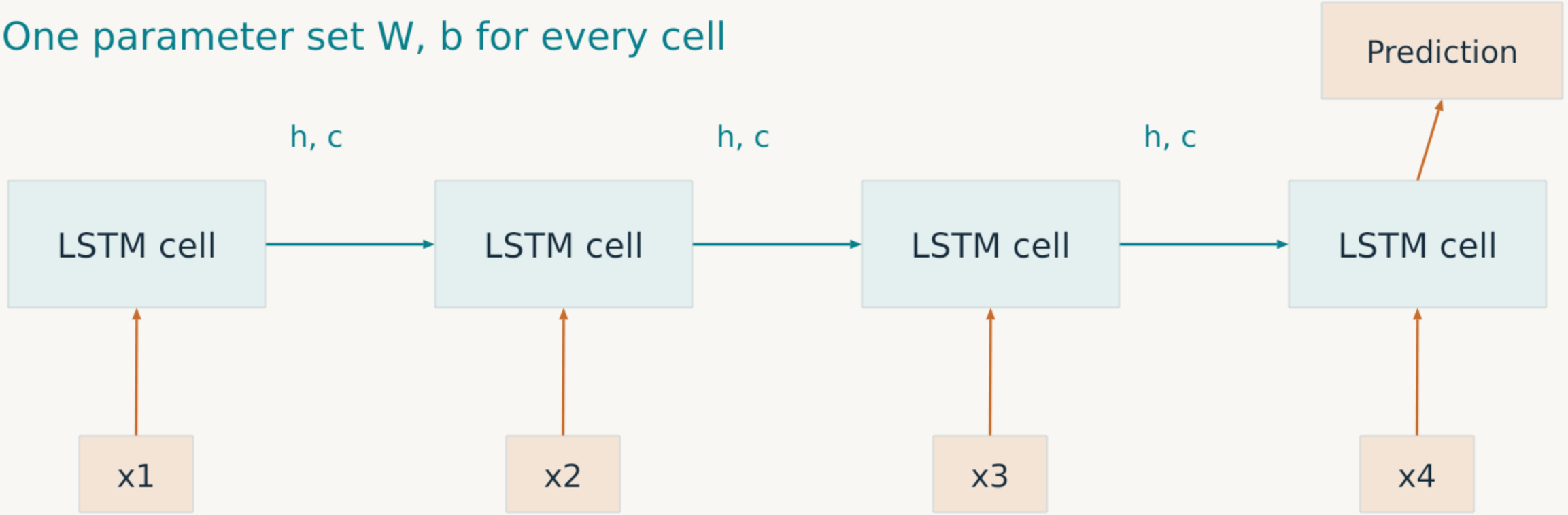
t = 1	t = 2	t = 3	t = 4	t = 5
0.2	0.9	−0.1	−0.4	0.7
marker = 0	marker = 1	marker = 0	marker = 1	marker = 0

Last marked value = −0.4. Classification target = nonpositive.

Recurrence reuses one learned rule

The same cell transforms each input and the state carried from the previous time.

One parameter set W, b for every cell



Weights are shared over time. Each independent sequence starts with $h_0 = 0$ and $c_0 = 0$.

Cell state and hidden state

The cell state carries memory. The hidden state exposes a gated version of that memory.

Cell state c_t

H memory values per example

Retain part of the old state.
Add selected candidate information.

The stored state is not bounded by tanh.

Hidden state h_t

H exposed values per example

Combine $\tanh(c_t)$ with an output gate.
Feed the next recurrent step.

The task head reads the final valid h.

The four gate computations

Sigmoid gates lie between 0 and 1. The candidate uses tanh.

$$q_t = \text{concat}(x_t, h_{t-1})$$

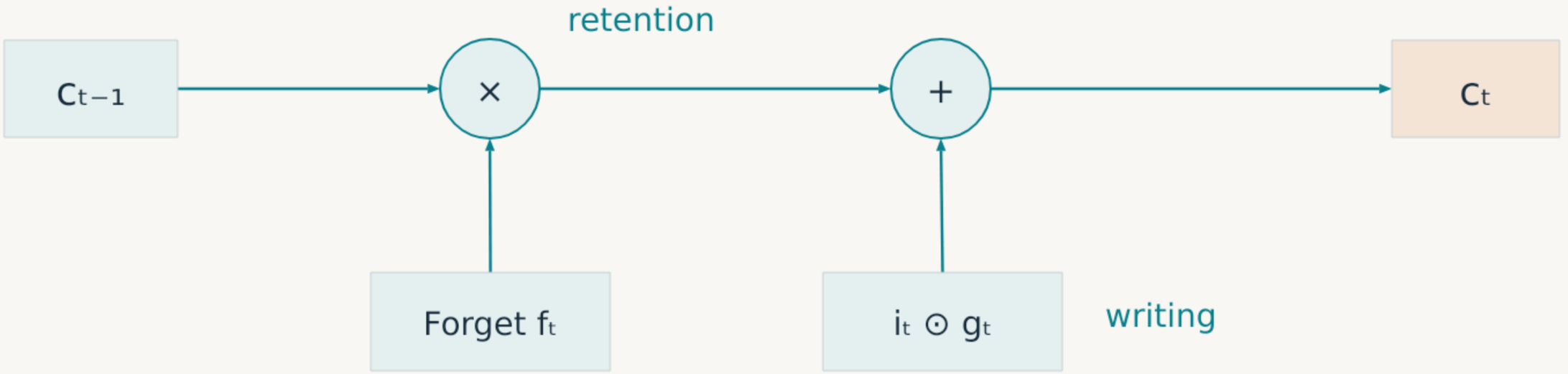
Input gate i	Forget gate f	Candidate g	Output gate o
σ: how much to write	σ: how much to retain	tanh: proposed content	σ: how much to expose

$$[i_t, f_t, g_t, o_t] = [\sigma, \sigma, \tanh, \sigma](q_t W + b)$$

W has shape (D+H) × 4H. Each gate or candidate has H values.

The cell update combines retention and writing

All multiplications marked ⊙ act element by element.



$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot \tanh(c_t)$$

Retention and writing add together. Output gating controls what the hidden state exposes.

One memory coordinate calculated by hand

A scalar example separates the gate roles from matrix notation.

$c_{t-1} = 0.8 \quad f = 0.9 \quad i = 0.2 \quad g = -0.5$

$c_t = 0.9(0.8) + 0.2(-0.5) = 0.62$

$h_t = 0.7 \tanh(0.62) \approx 0.386$

Forget gate retains 0.72.

Input gate writes −0.10.

Output gate exposes part of the updated memory.

Layers, directions, and parameter counts

Time steps reuse weights. Stacked layers introduce additional weights.

Component	Formula or behavior	Lab choice
One LSTM layer	4H(D+H+1) parameters	D=2, H=16: 1,216
Classification head	HK + K parameters	K=2: 34
Total classification model	Recurrent layer + head	1,250 parameters
Stacked / bidirectional	Extra layers / reverse context	Discussed extensions

Sequence length changes compute and activation storage, not the number of learned weights.

Labels determine the output pattern

A sequence can have one target, one target per time, or an output sequence.

Pattern	Label shape	Example
Many-to-one classification	[B], integer class IDs	Sign of the last marked value
Many-to-one regression	[B], real numbers	Last marked value
Aligned sequence tagging	[B,T], plus label mask	Event type at each time
Sequence-to-sequence	[B,Tout,...]	Future trajectory or translated sequence

The laboratory implements the two many-to-one tasks.

Numerical, logical, and categorical inputs

The feature schema remains fixed across training and prediction.

Value type	Encoding	Important distinction
Numerical measurement	Float, training-only scaling	Preserve units in metadata
Logical marker	0 or 1	Separate from padding validity
Category or token	One-hot or learned embedding	Define vocabulary and unknown values
Missing observation	Imputation + optional indicator	Different from an absent time step
Label	Class ID or finite scalar	One aligned target per sequence

Lab arrays: X [B,T,2], mask [B,T], y [B].

Variable length, padding, and minibatches

A mask prevents padded time steps from changing either recurrent state.

Sequence	t=1	t=2	t=3	t=4	t=5
Mask A	1	1	1	1	1
Mask B	1	1	1	0	0
Mask C	1	1	0	0	0

$$c = \text{mask} \times c_{new} + (1 - \text{mask}) \times c_{prev}$$

$$h = \text{mask} \times h_{new} + (1 - \text{mask}) \times h_{prev}$$

Shuffle independent sequences. Preserve time order within every sequence.

Time splits and window leakage

A valid test set reflects what would be available at the intended prediction time.

Independent sequences

Random split is appropriate here.

70% train / 15% validation / 15% test

Scaling uses valid training time steps.

Real temporal observations

Use chronological or rolling splits.

Keep overlapping windows and related entities under control.

Record the forecast horizon and cutoff.

Classification and regression heads

A linear head reads the final valid hidden state.

$$z = h_{final} W_o + b_o$$

Classification: $L = -\log \text{softmax}(z)_{true}$

Regression: $L = \text{mean}((\hat{y} - y)^2)$

Class target: sign of last marked value

Regression target: last marked value

Target scaling uses training labels only.

The training loop, learning rate, and AdamW

A minibatch update accumulates gradients across all valid times before changing the shared weights.

```
for ids in shuffled_batches:
    loss, grads = net.loss_grad(
        sequences, labels, task)
    optimizer.step(net.params, grads)

# AdamW: lr=0.003, weight_decay=0.0001
# Global gradient norm cap: 5
# Restore lowest-validation-loss weights.
```

Batch size: 32
Maximum epochs: 80
Early-stop patience: 15

Tune learning rate and hidden size on validation examples.

Full runnable source and tests accompany the presentation.

Backpropagation through time

The gradient follows both the hidden-state path and the cell-state path in reverse time.

Direct cell path: $\partial c_t / \partial c_{t-1}$ includes f_t

Sigmoid derivative: $a(1-a)$ Tanh derivative: $1-a^2$

$\partial L/\partial W$ = sum of contributions over time

Reverse the time loop.

Combine hidden and cell gradients.

Respect mask identity paths.

Apply one update after the batch.

An LSTM step in executable NumPy

The full source includes forward caching and the matching reverse-time derivative.

```
# mask: [B,1] for the current time step
q = np.concatenate([x_t, h], axis=1)
vi, vf, vg, vo = np.split(q @ W + b, 4, axis=1)
i, f = sigmoid(vi), sigmoid(vf)
g, o = np.tanh(vg), sigmoid(vo)

c_new = f * c + i * g
h_new = o * np.tanh(c_new)
c = mask * c_new + (1-mask) * c
h = mask * h_new + (1-mask) * h
```

Input dimension D = 2
Hidden dimension H = 16
Gate order: i, f, g, o

1,250 parameters for 2 classes
1,233 for scalar regression

Full runnable source and tests accompany the presentation.

Training, testing, and saved-model prediction

The command-line harness records splits, preprocessing, weights, and measured errors.

```
python run_all.py

python lab.py train --data data/classification.npz \
    --out runs/experiment_2 --lr 0.001
python lab.py evaluate --data data/classification.npz \
    --run runs/experiment_2
```

Batch training

Forward → loss → backward
AdamW update → validation

Saved checkpoint

Weights and feature statistics
Split indices and dataset hash

Independent testing

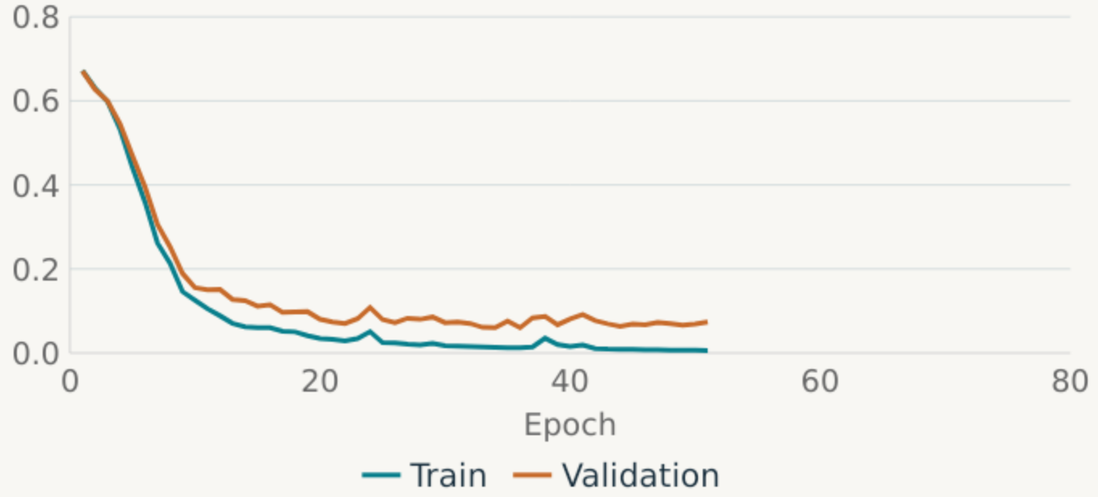
Restore selected weights.
Report held-out metrics.

Data archive: X, mask, y. Prediction archives omit y.

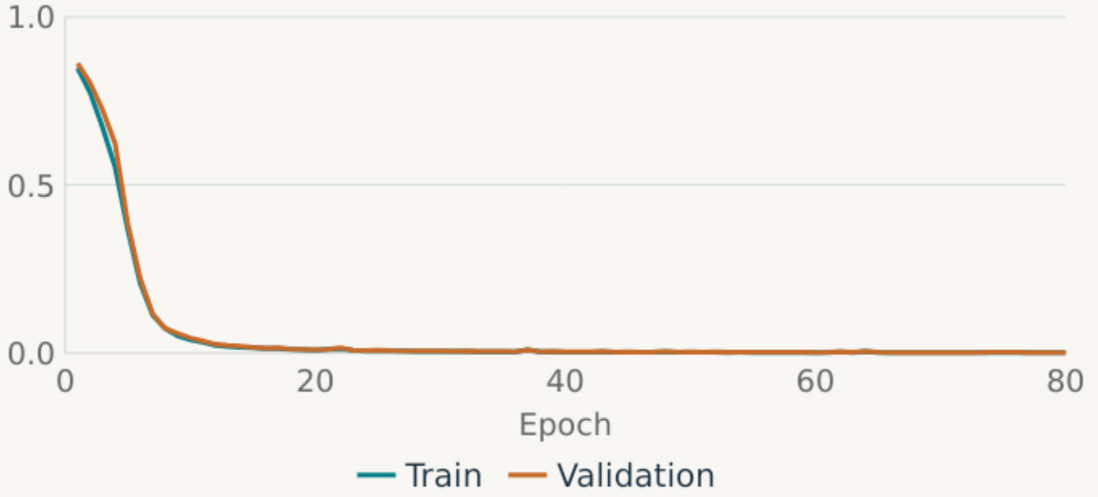
Measured results on held-out sequences

Reference runs test whether the network remembers the last marked value.

Classification: cross-entropy



Regression: standardized MSE



Test accuracy 98.35%

Majority baseline: 50.00%
Macro-F1 0.9835
242 test examples; best epoch 36

Test RMSE 0.0211

Training-mean baseline: 0.5752
MAE 0.0161 R^2 0.9987
240 test examples; best epoch 80

Synthetic teaching tasks. These scores do not establish real-world performance.

Diagnostics and meaningful tests

Sequence models need checks for state handling, masking, leakage, and dependency length.

Observation	Possible cause	Targeted check
Prediction changes with extra padding	States or gradients ignore the mask	Padding-invariance test
Performance fails on longer sequences	Dependency length exceeds training experience	Evaluate by length and marker delay
Training becomes unstable	Gradient growth or excessive learning rate	Norm logging, clipping, smaller step
Excellent test score on overlapping windows	Temporal or entity leakage	Chronological / group split audit

Seven tests passed. Full BPTT on short CPU sequences keeps the arithmetic inspectable.

Laboratory and further reading

The notes explain every equation and the lab exposes each trainable operation.

Exercises

1. Derive the 1,250-parameter count.
2. Compare with a plain recurrent network.
3. Vary sequence length and marker delay.
4. Add a masked output at every valid step.

Primary references

Hochreiter & Schmidhuber, LSTM
Gers et al., the forget gate
Modern LSTM equations
Deep Learning, Chapter 10

Companion lab: source code, automated tests, datasets, reference checkpoints, training histories, test metrics, prediction examples, and complete lecture notes.